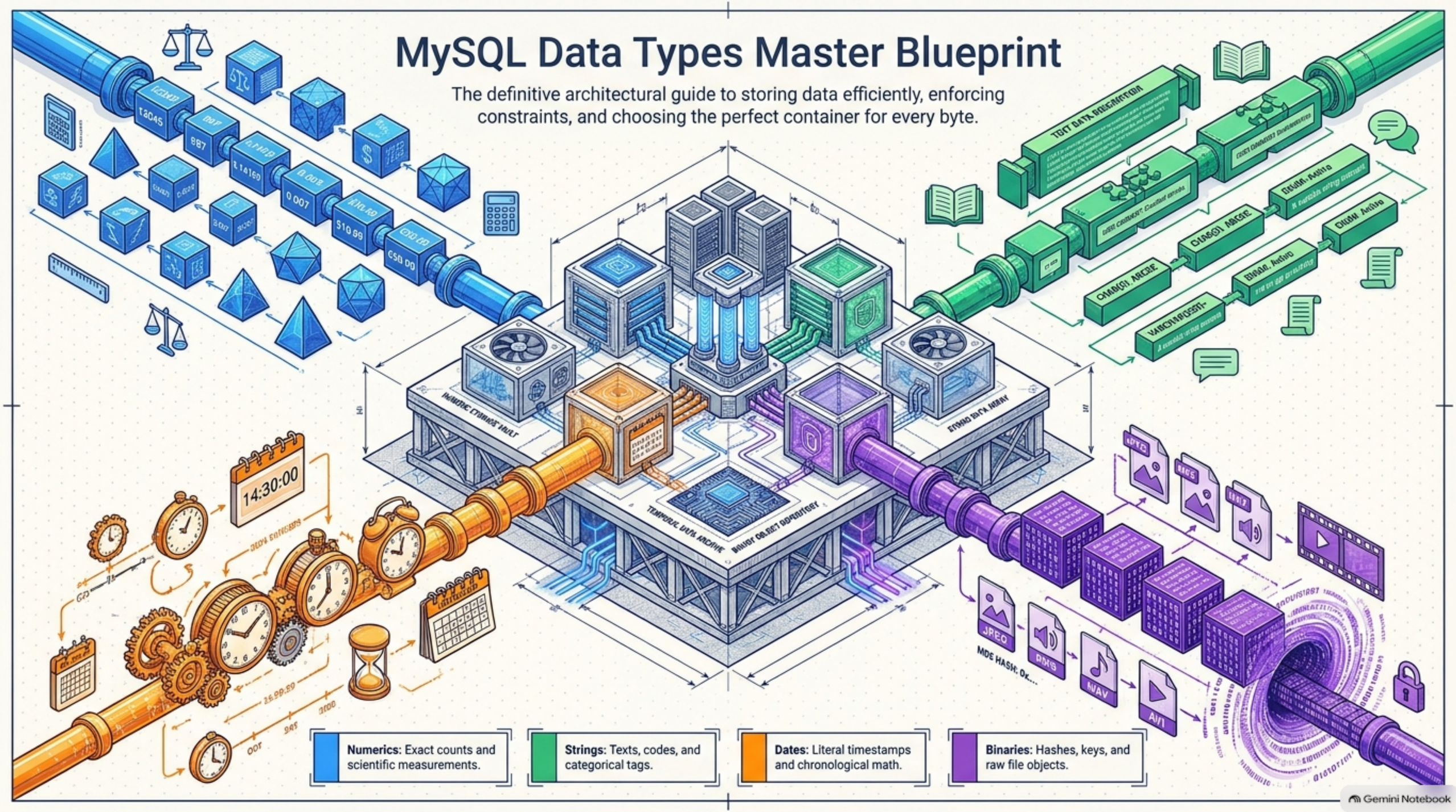
MySQL Data Types Master Blueprint

The definitive architectural guide to storing data efficiently, enforcing constraints, and choosing the perfect container for every byte.



Numerics: Exact counts and scientific measurements.

Strings: Texts, codes, and categorical tags.

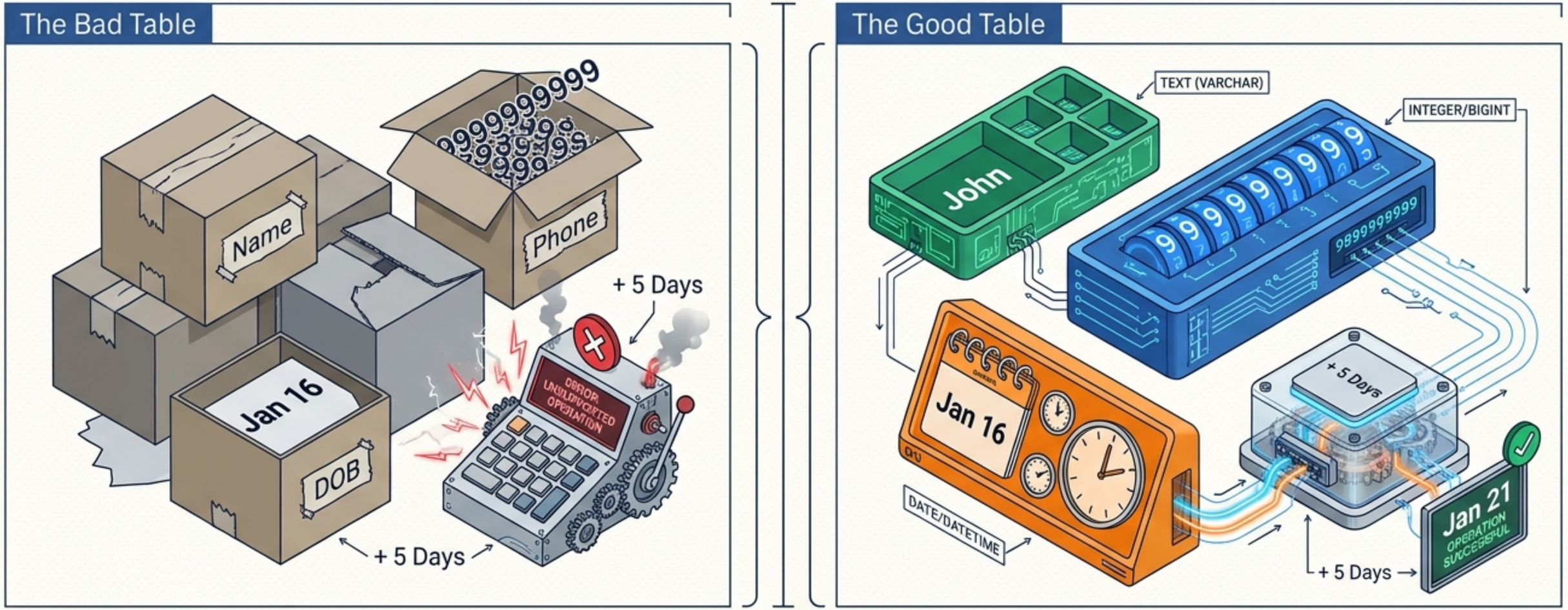
Dates: Literal timestamps and chronological math.

Binaries: Hashes, keys, and raw file objects.

Gemini Notebook

Why Types Matter: Efficiency and Constraints

Databases aren't text files; they are strict mechanical engines.
We define types for two reasons:

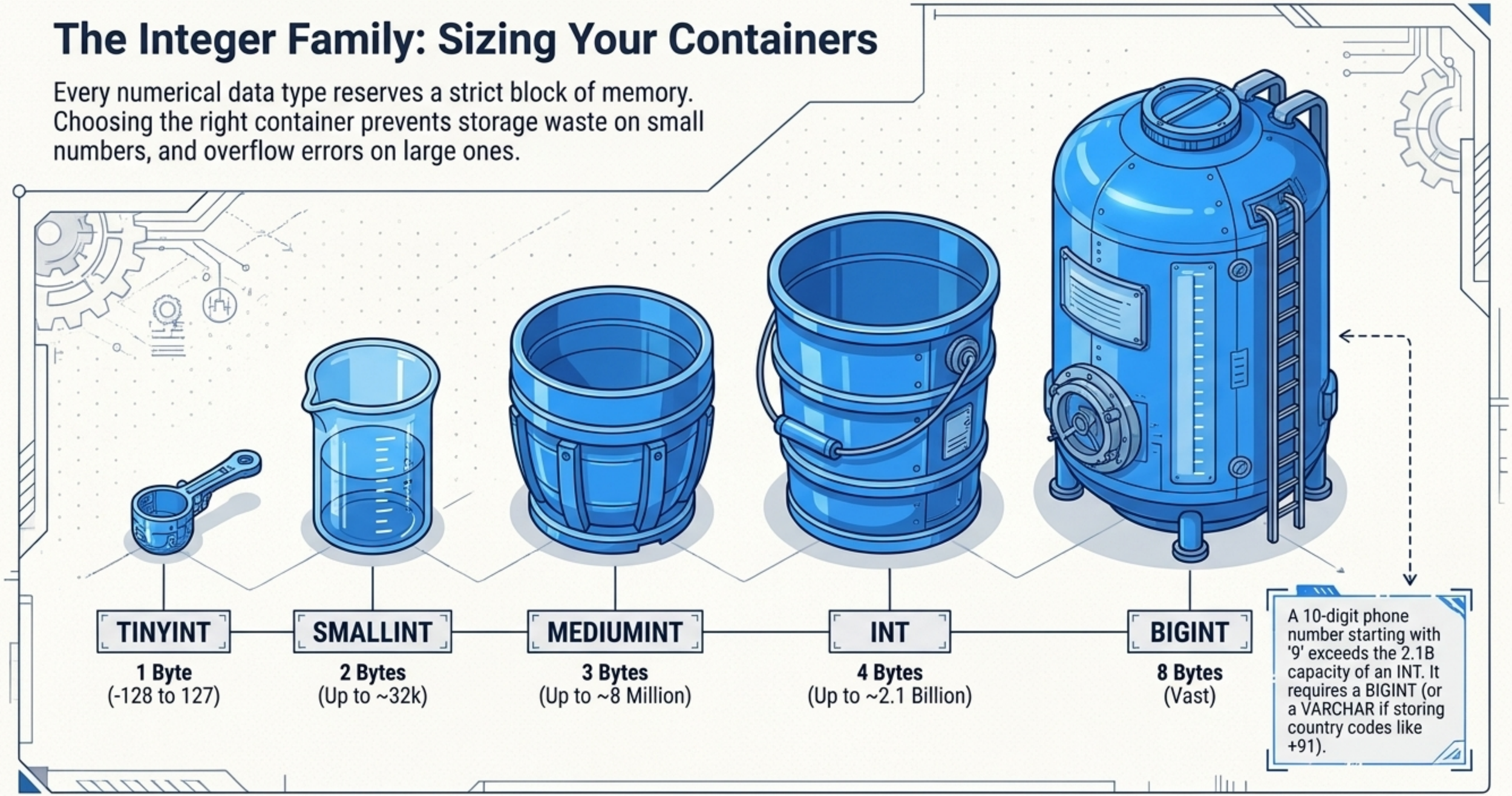


1. **Storage Efficiency:** Don't use a bucket when a spoon will do.
2. **Functional Constraints:** You cannot perform mathematical operations or timezone shifts on raw text.

Gemini Notebook

The Integer Family: Sizing Your Containers

Every numerical data type reserves a strict block of memory. Choosing the right container prevents storage waste on small numbers, and overflow errors on large ones.



TINYINT
1 Byte
(-128 to 127)

SMALLINT
2 Bytes
(Up to ~32k)

MEDIUMINT
3 Bytes
(Up to ~8 Million)

INT
4 Bytes
(Up to ~2.1 Billion)

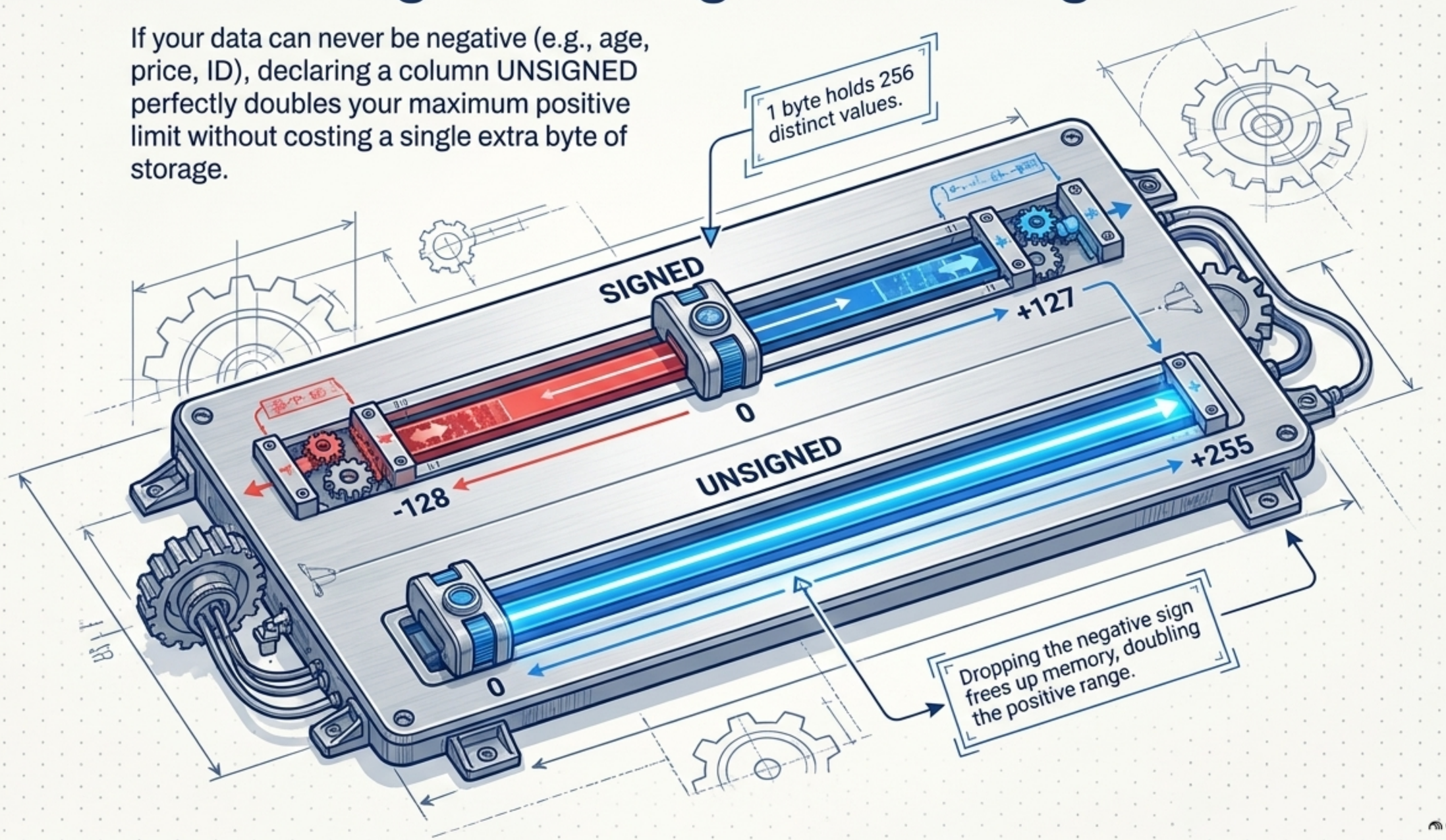
BIGINT
8 Bytes
(Vast)

A 10-digit phone number starting with '9' exceeds the 2.1B capacity of an INT. It requires a BIGINT (or a VARCHAR if storing country codes like +91).

Gemini Notebook

The Storage Hack: Signed vs. Unsigned

If your data can never be negative (e.g., age, price, ID), declaring a column UNSIGNED perfectly doubles your maximum positive limit without costing a single extra byte of storage.

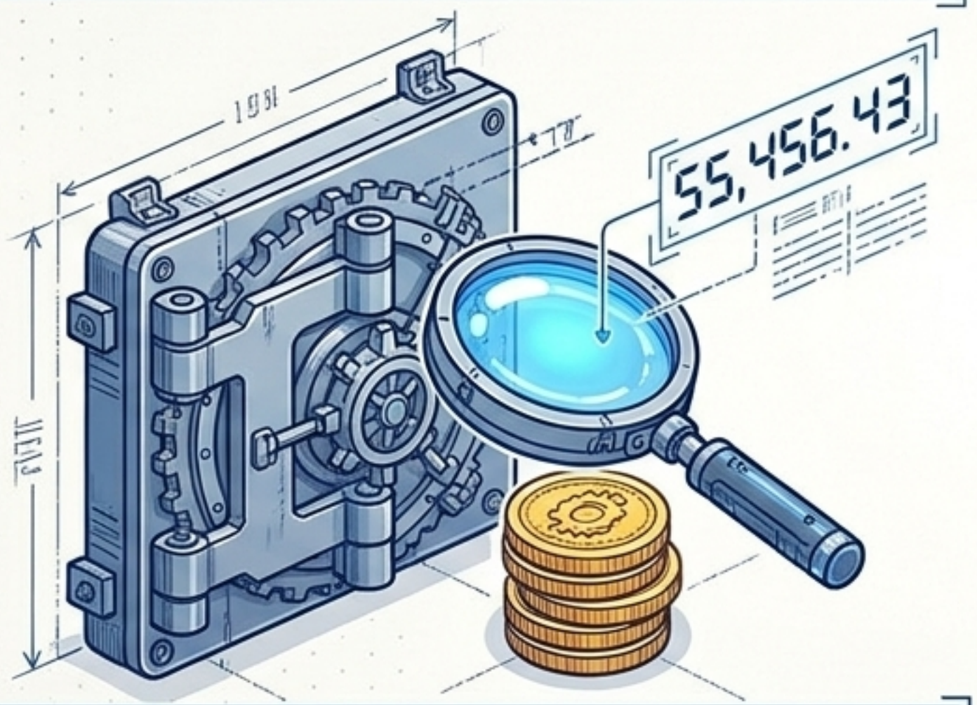


Gemini Notebook

Fractions: Exact Calculations vs. Infinite Scale

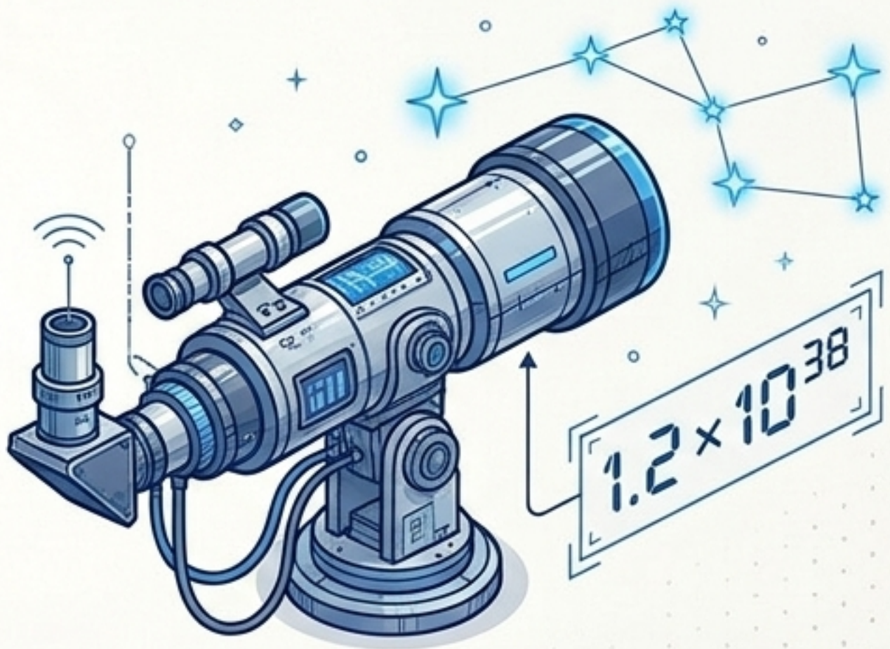
Floating-point numbers “float” their decimal to store massive scales (like the distance between stars) but introduce microscopic rounding errors. Decimals store exact digits at the cost of processing speed.

The Banker (Fixed Point / DECIMAL)



Trait:	Absolute precision.
Use Case:	Financial calculations, exact balances.

The Astronomer (Floating Point / FLOAT & DOUBLE)



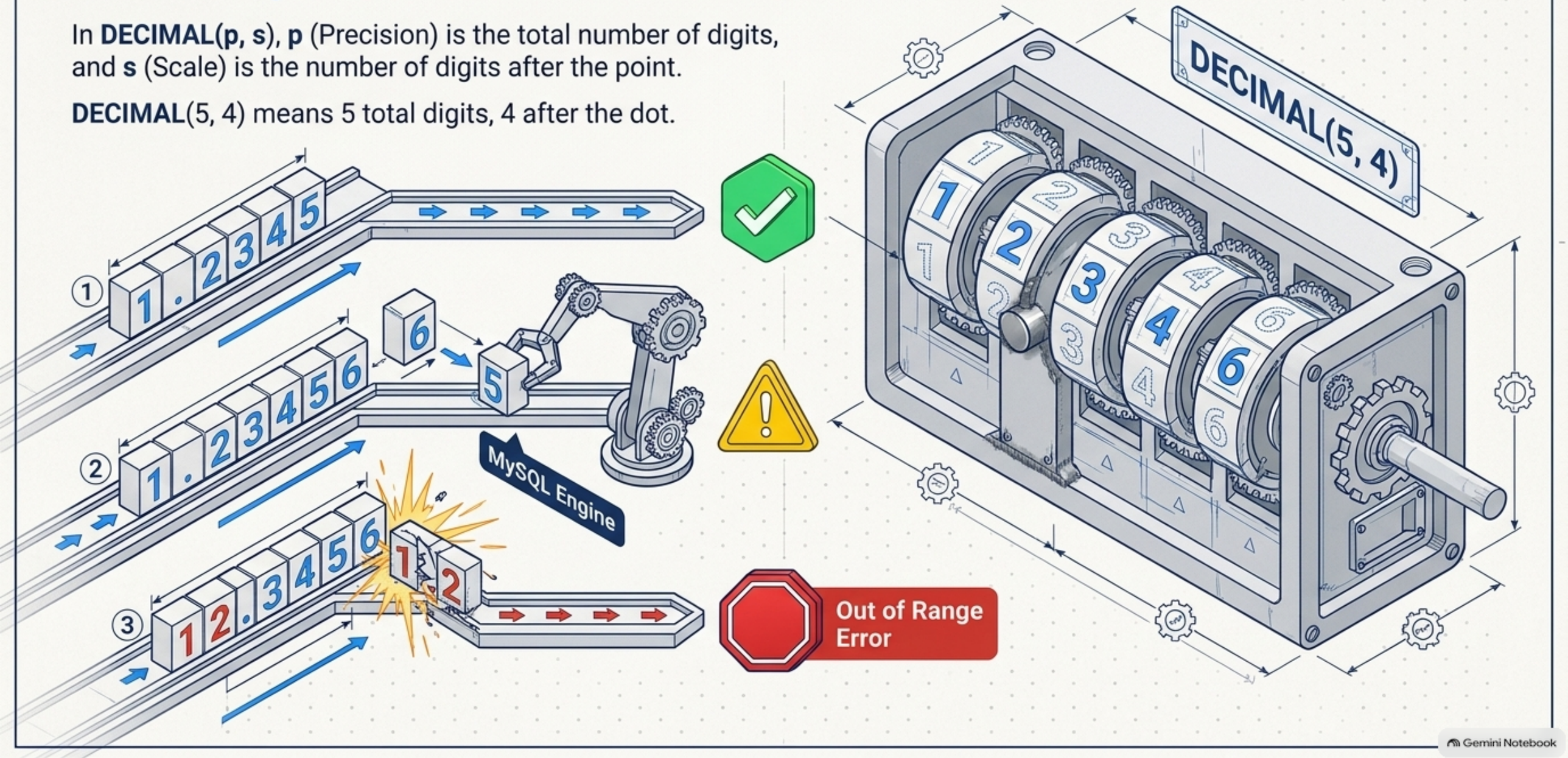
Trait:	Approximate scaling.
Use Case:	Scientific data where extreme bounds matter more than the 15th decimal place.

Gemini Notebook

Anatomy of a Decimal: Precision & Scale

In **DECIMAL(p, s)**, **p** (Precision) is the total number of digits, and **s** (Scale) is the number of digits after the point.

DECIMAL(5, 4) means 5 total digits, 4 after the dot.

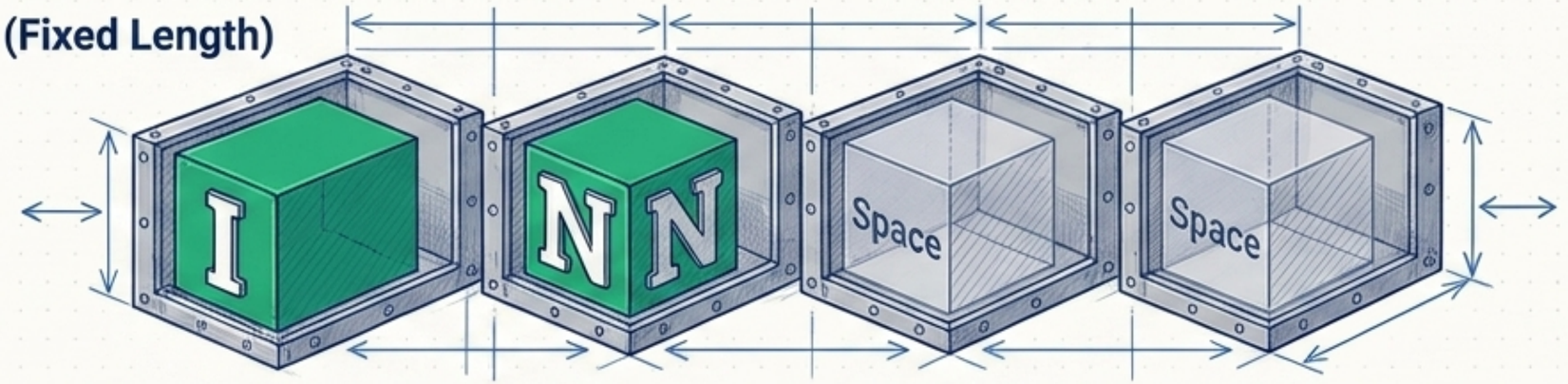


Gemini Notebook

String Storage: Fixed vs. Variable Logic

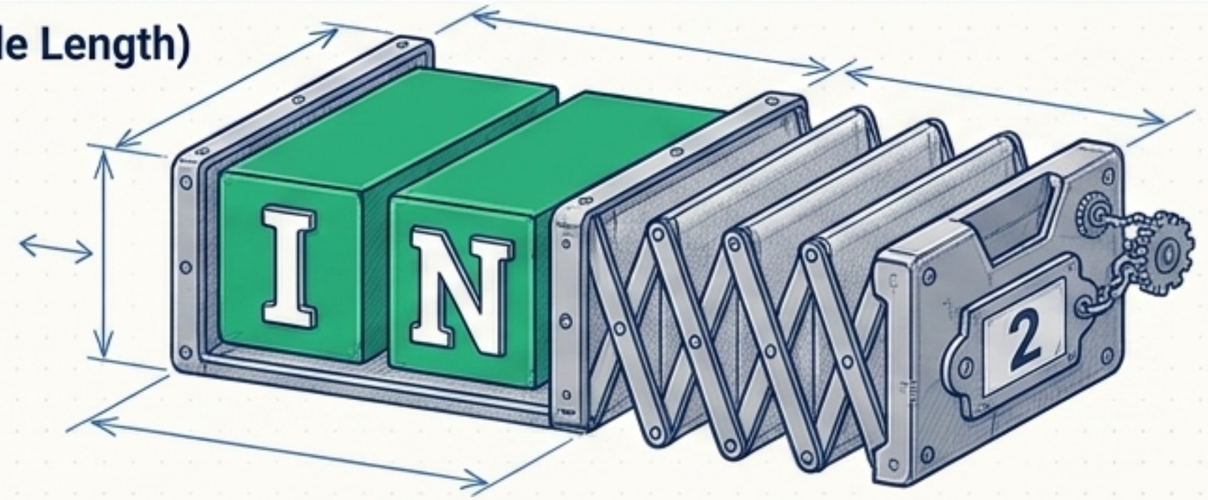
CHAR provides raw speed because the database always knows exactly where the next row begins.
VARCHAR saves storage space for variable data but requires an extra byte just to record the length of the string.

CHAR (Fixed Length)



Best for predictable, fixed data (e.g., State Codes). System pads empty space.

VARCHAR (Variable Length)

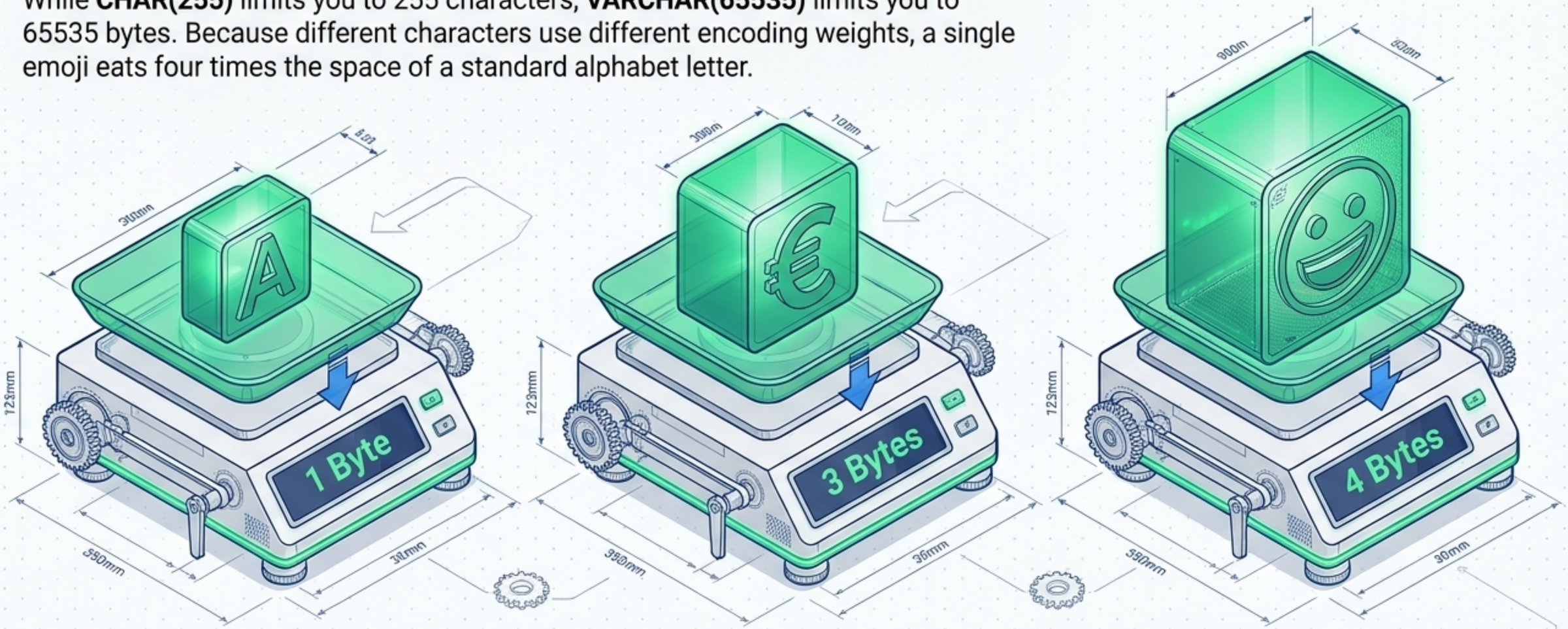


Takes exact space needed, plus a tiny 1-2 byte tag to track string length.

Gemini Notebook

The Weight of a Character

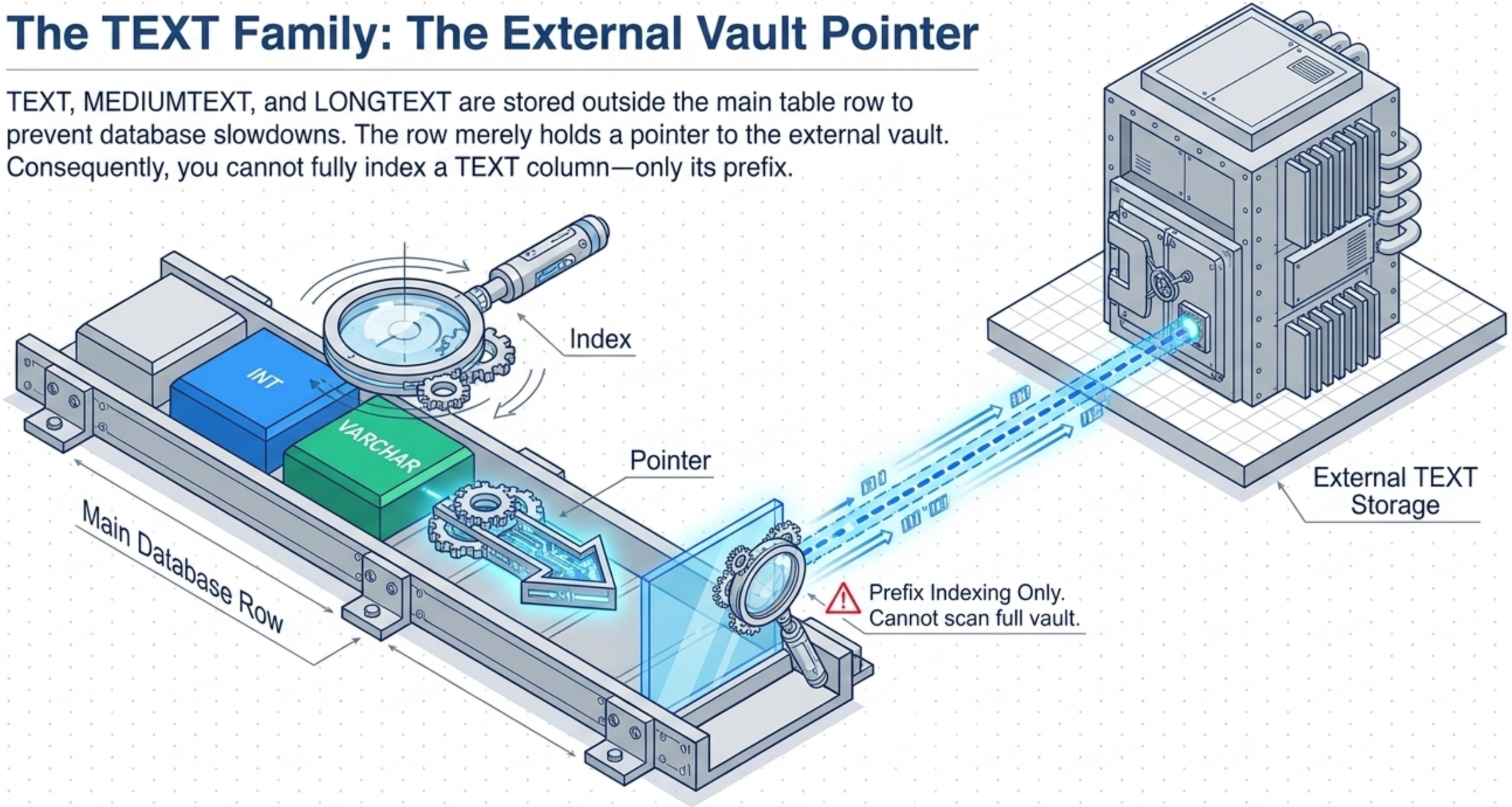
While **CHAR(255)** limits you to 255 characters, **VARCHAR(65535)** limits you to 65535 bytes. Because different characters use different encoding weights, a single emoji eats four times the space of a standard alphabet letter.



Gemini Notebook

The TEXT Family: The External Vault Pointer

TEXT, MEDIUMTEXT, and LONGTEXT are stored outside the main table row to prevent database slowdowns. The row merely holds a pointer to the external vault. Consequently, you cannot fully index a TEXT column—only its prefix.

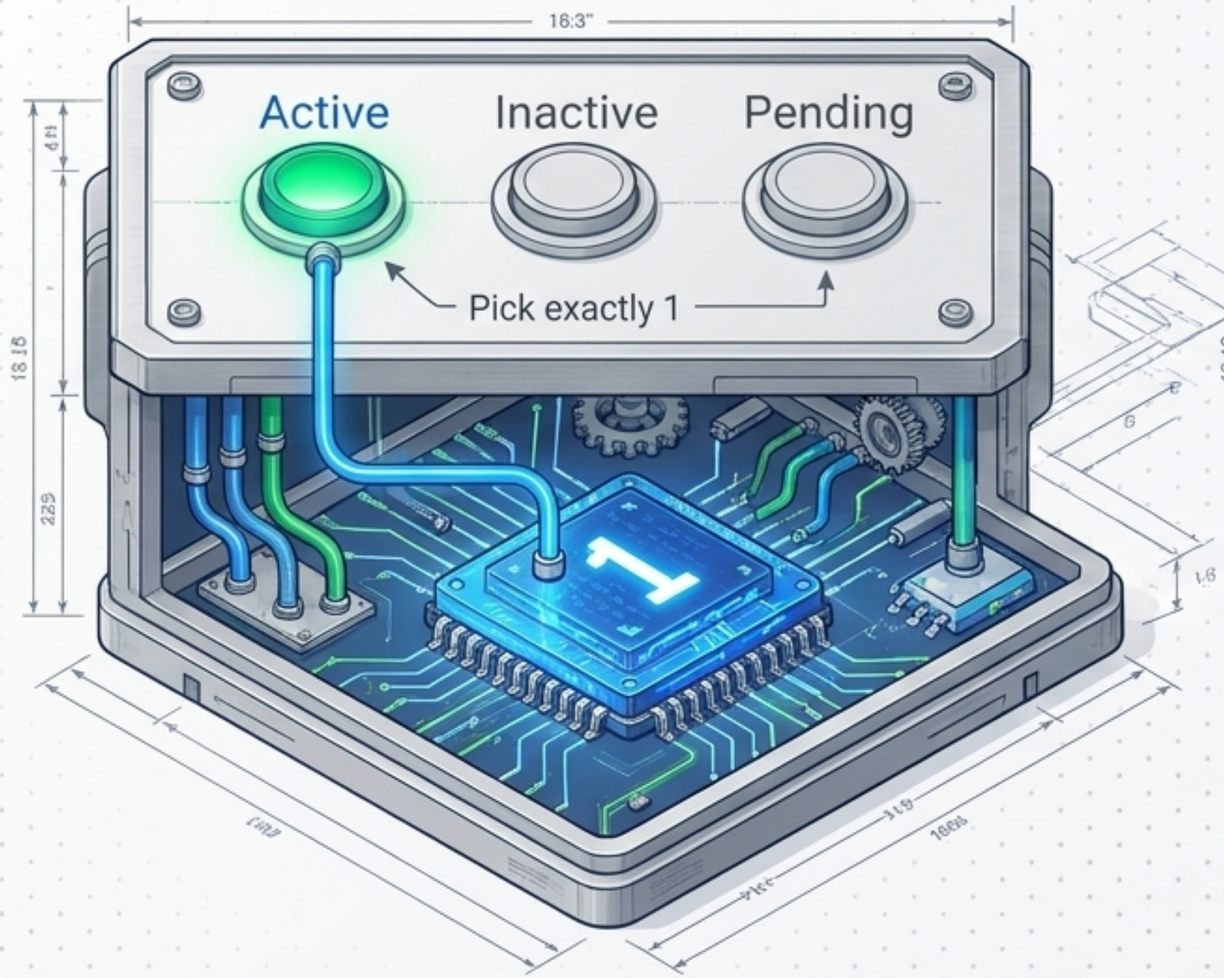


Gemini Notebook

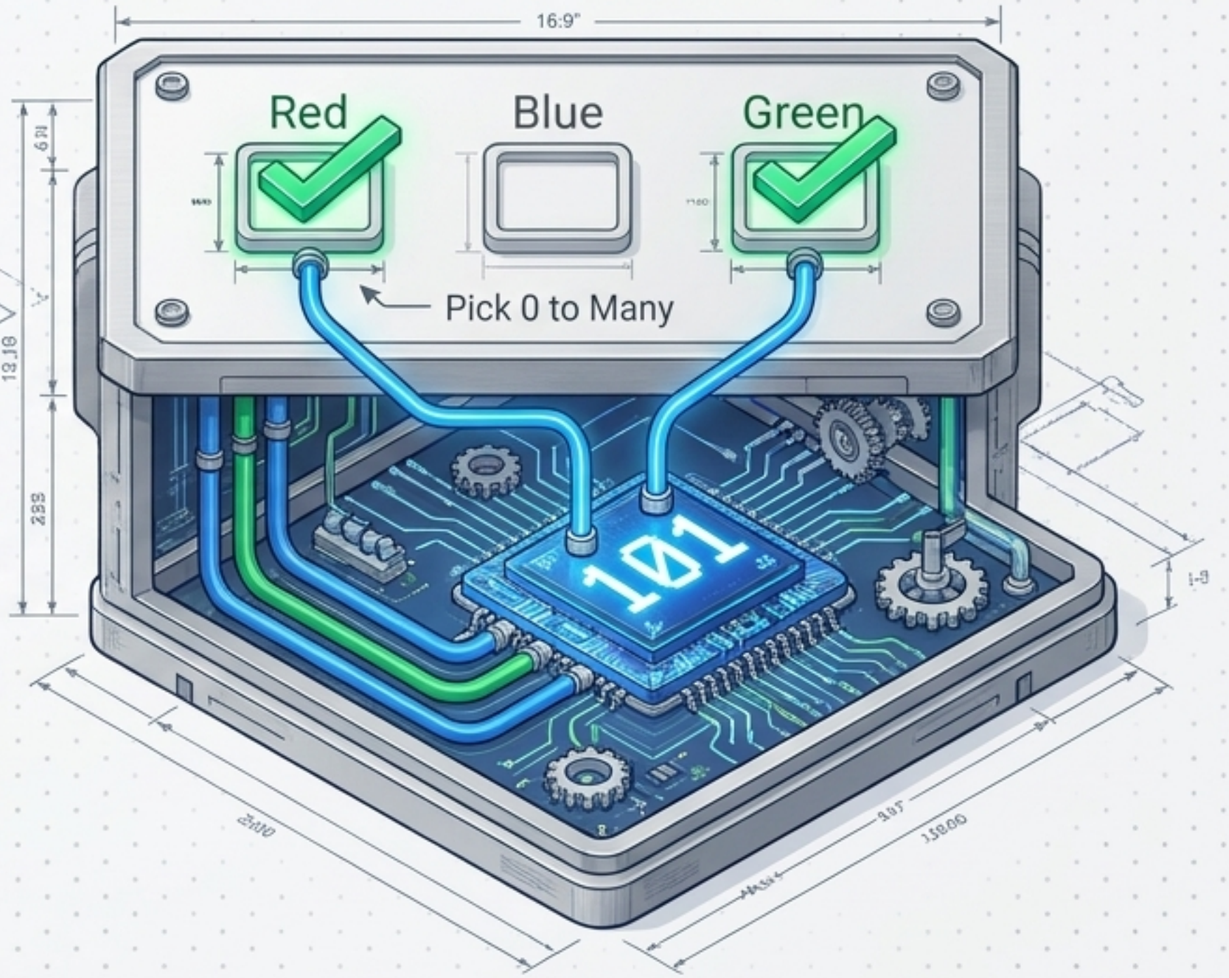
Enforcing State: ENUM vs. SET Categories

Both ENUM and SET restrict inputs to a predefined list. While you configure them with text, MySQL secretly stores them as **tiny integers** in the metadata, making them **incredibly lightweight** to search and store.

ENUM (The Radio Button)



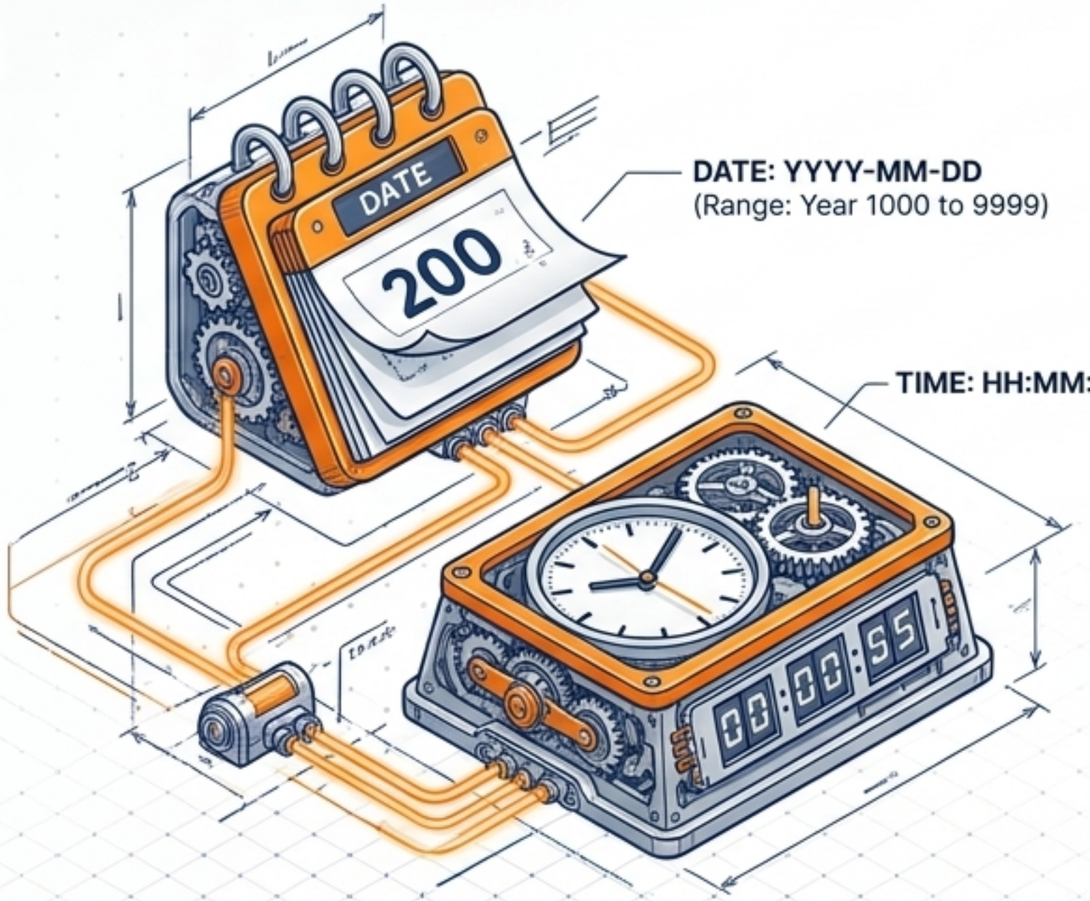
SET (The Checkbox)

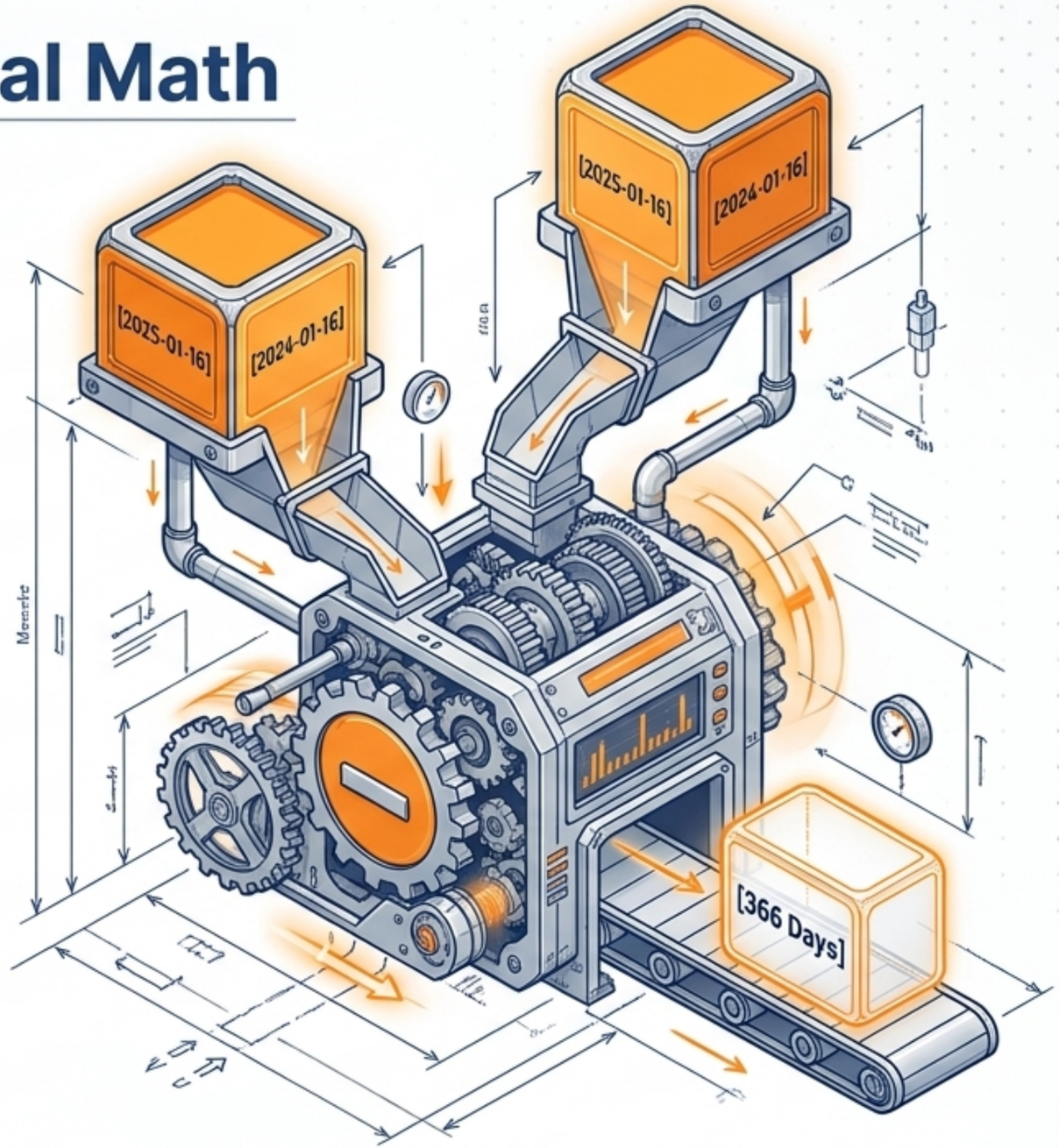


Gemini Notebook

Time Travel & Chronological Math

Never store dates as **VARCHAR**. Native date types allow the database engine to enforce chronological constraints and perform high-speed date math—**calculating age, finding ranges, or extracting the month instantly.**

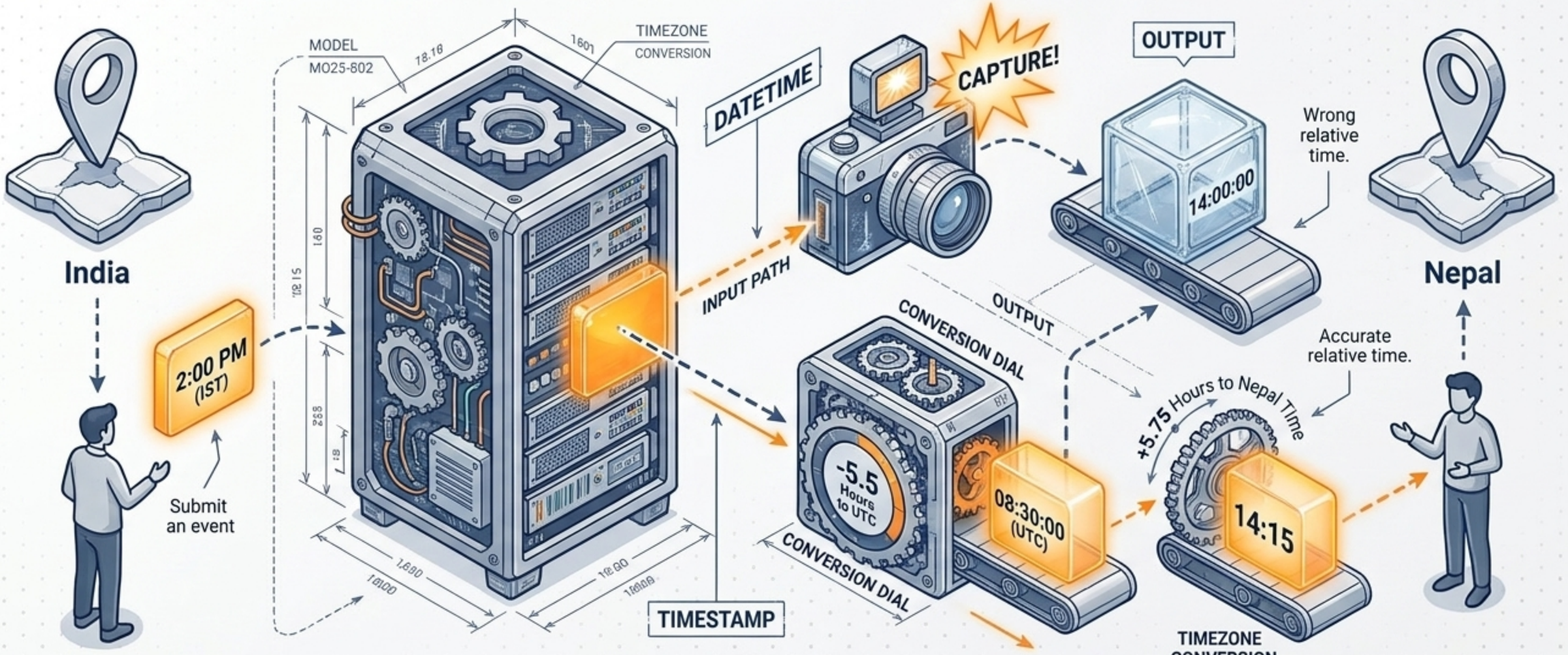




Gemini Notebook

The Timezone Shift: DATETIME vs. TIMESTAMP

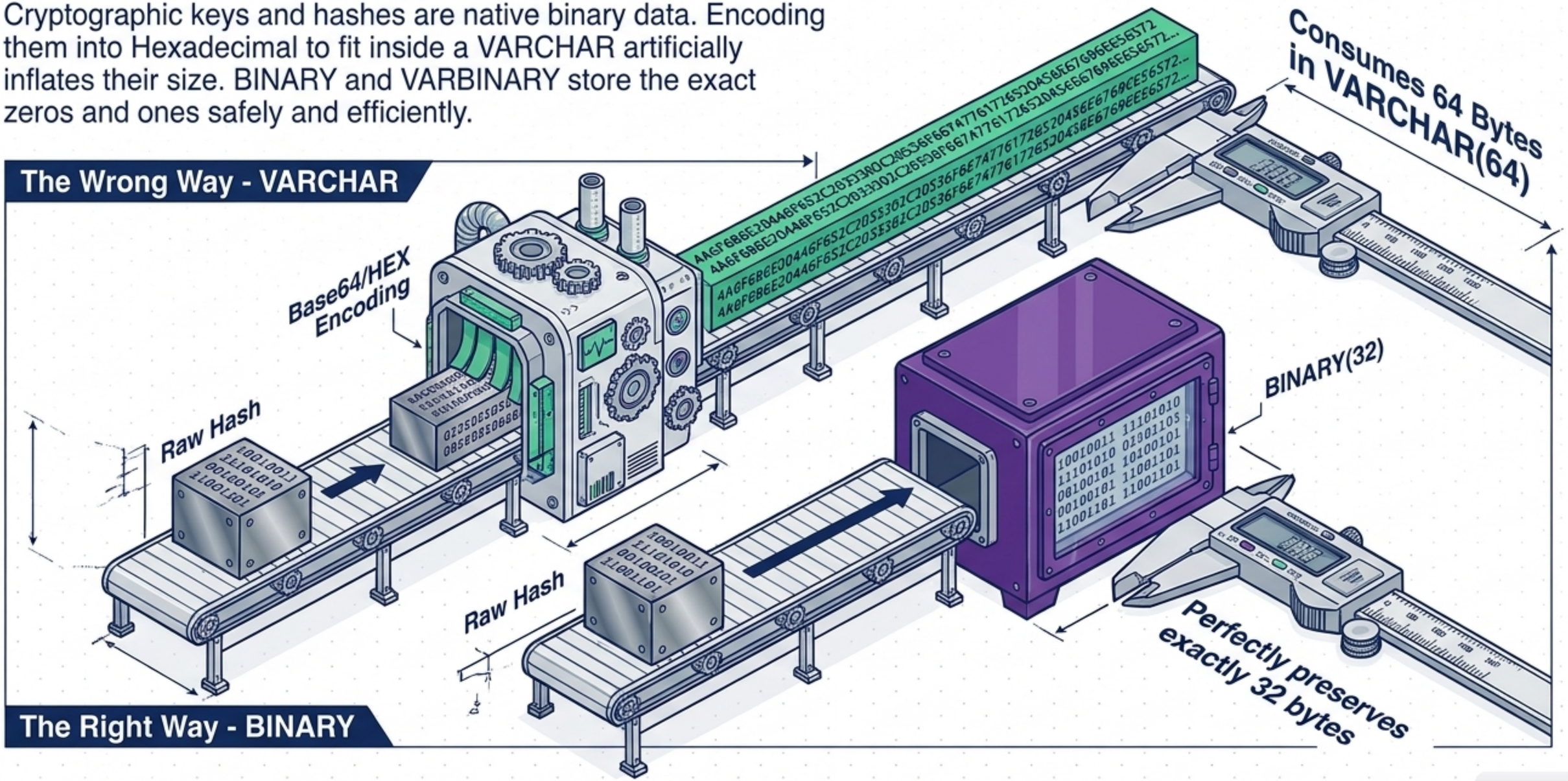
DATETIME acts like a photograph of a clock—it never changes. **TIMESTAMP** translates every input into UTC (Coordinated Universal Time), automatically adapting the display based on the retrieving server's timezone.



Gemini Notebook

Raw Precision: Storing Exact Hashes & Keys

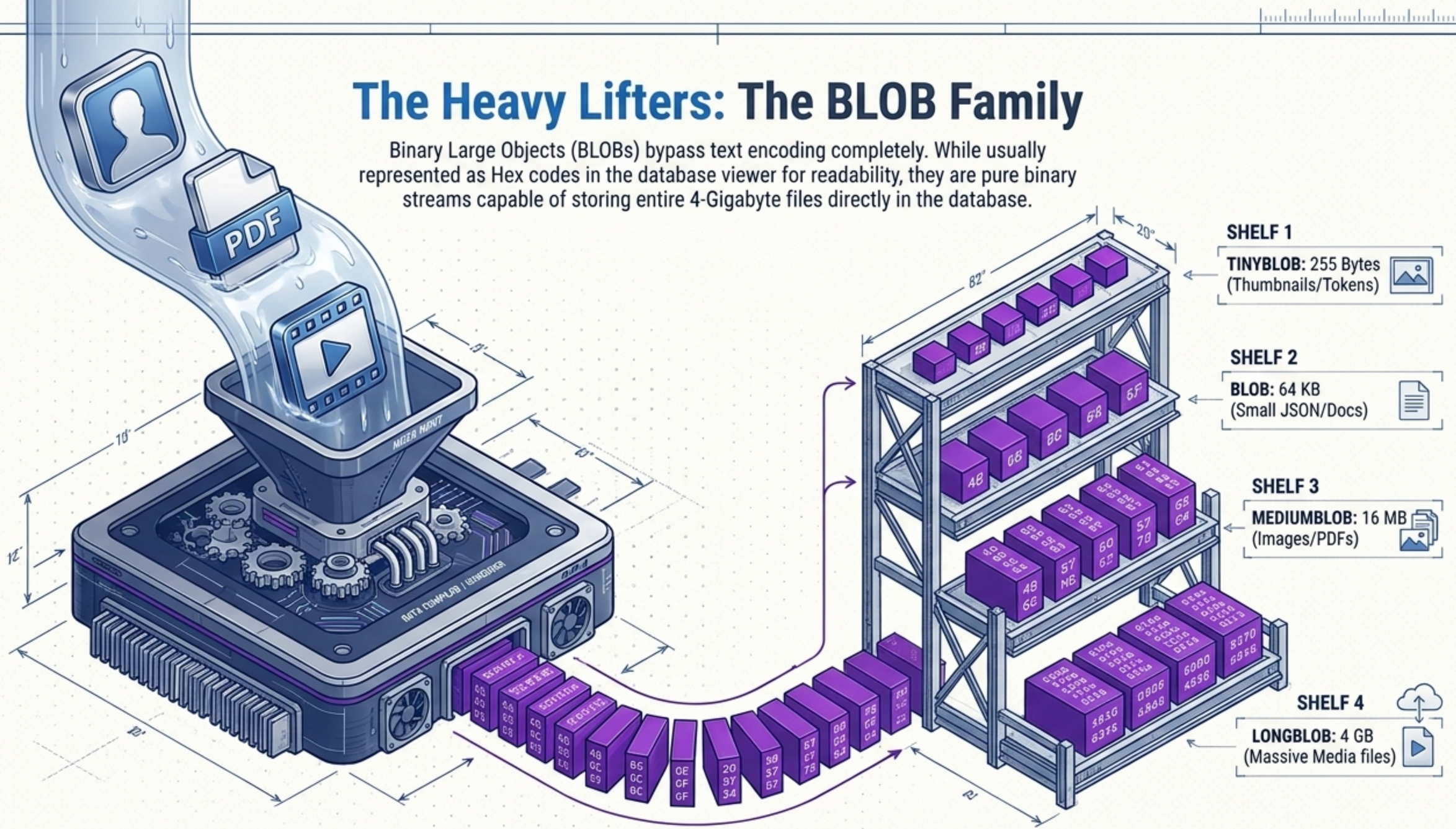
Cryptographic keys and hashes are native binary data. Encoding them into Hexadecimal to fit inside a VARCHAR artificially inflates their size. BINARY and VARBINARY store the exact zeros and ones safely and efficiently.



Gemini Notebook

The Heavy Lifters: The BLOB Family

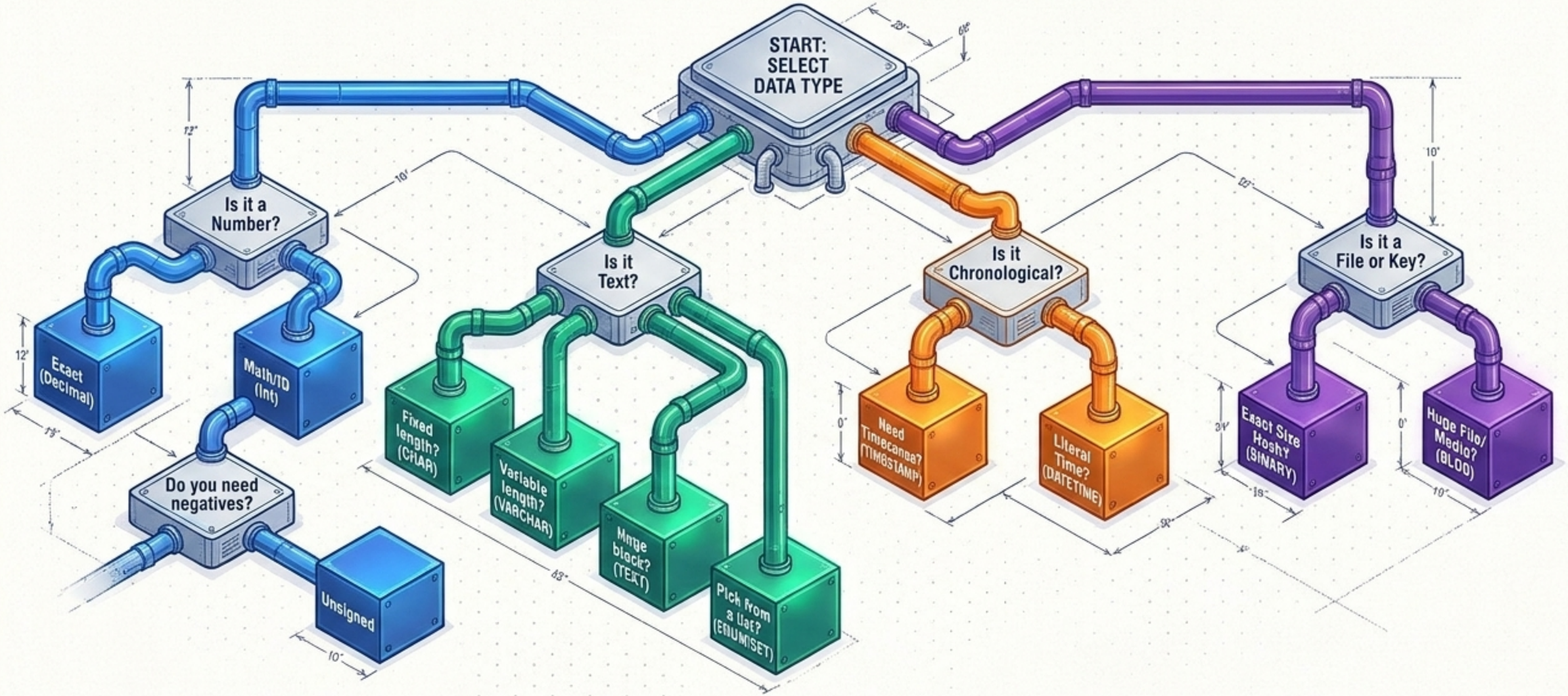
Binary Large Objects (BLOBs) bypass text encoding completely. While usually represented as Hex codes in the database viewer for readability, they are pure binary streams capable of storing entire 4-Gigabyte files directly in the database.



Gemini Notebook

The Data Type Decision Matrix

Every byte matters. Use this blueprint to architect tables that are strict enough to maintain data integrity, yet optimized enough for maximum query speed. Right tool, right job.



Gemini Notebook